

7. Transformer (BERT~)

BERT (Bidirectional Encoder Representations from Transformers)

사전 학습 방법

1. MLM (Masked Language Modeling, 마스크된 언어 모델링)
2. NSP (Next Sentence Prediction, 다음 문장 예측)

특수 토큰

입력 임베딩 구조

Fine-tuning

모델 실습

BART (Bidirectional Auto-Regressive Transformer)

구조

사전 학습 방법

1. 토큰 마스크 (Token Masking)
2. 토큰 삭제 (Token Deletion)
3. 문장 순열 (Sentence Permutation)
4. 문서 회전 (Document Rotation)
5. 텍스트 채우기 (Text Infilling)

미세 조정 방법

1. 문장 분류
2. 토큰 분류
3. 문장 생성
4. 기계 번역

모델 실습

ELECTRA

핵심 아이디어

GAN과의 차이점

BERT 대비 장점

사전 학습 방법 : RTD (Replaced Token Detection)

가중치 공유 전략

파인튜닝

모델 실습

T5 (Text-to-Text Transfer Transformer)

핵심 특징: Text-to-Text 구조

구조

사전 학습 방법

미세 조정 (Fine-tuning)

모델 실습

BERT (Bidirectional Encoder Representations from Transformers)

2018년 구글이 발표한 트랜스포머 기반 자연어 처리 모델

핵심 특징 : 양방향(Bidirectional) 인코더

- 기존 언어 모델은 문장을 좌→우 순차적으로 학습해 이전 단어만 참조 가능했음.
- BERT는 입력 시퀀스를 양쪽 방향에서 처리해 이전 + 이후 단어를 모두 참조하므로 단어의 의미와 문맥을 더 정확하게 파악함.

구조

- 트랜스포머의 **인코더 모듈만** 사용 (디코더 없음).

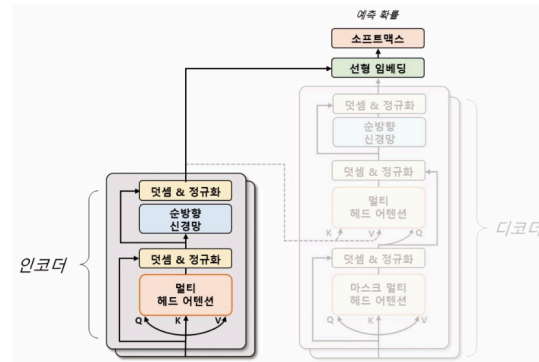


그림 7.13 트랜스포머와 BERT 모델 비교

- 인코더가 입력 문장의 단어들을 임베딩해 벡터화한 후, 좌우 양방향으로 처리해 문장 전체 의미를 추출함.

사전 학습 (Pre-training)

- 두 가지 방법 사용
 - **MLM (Masked Language Modeling):** 마스킹된 단어 예측
 - **NSP (Next Sentence Prediction):** 다음 문장 예측
- 대규모 데이터로 사전 학습되어 있어 전이 학습(Transfer Learning)에 주로 활용됨.

장점

- 적은 양의 데이터로도 높은 정확도 달성 가능
- 모델 학습 시간 대폭 단축
- 다양한 자연어 처리 작업에서 높은 성능

사전 학습 방법

1. MLM (Masked Language Modeling, 마스킹된 언어 모델링)

입력 문장에서 일부 단어를 [MASK]로 가리고 해당 단어를 예측하는 방식.

- e.g., 'I'm learning PyTorch' → 'I'm [MASK] PyTorch'에서 [MASK] 예측. 이 과정에서 양방향 문맥 정보를 참조하므로 문장 전체 의미를 이해하는 능력이 향상됨.

[MASK] 토큰 적용 규칙 (전체 토큰의 15%를 마스킹) :

- 80%: [MASK] 토큰으로 대체
- 10%: 어휘 사전의 무작위 단어로 변경
- 10%: 실제 토큰 그대로 유지

세 가지로 나누는 이유?

- 만약 항상 [MASK]로만 대체하면 모델이 "[MASK] 토큰이 있을 때만 예측하면 된다"고 학습하게 됨.
- 그런데 실제 파인튜닝이나 추론 시에는 입력 문장에 [MASK]가 없음.
- 즉 사전 학습과 실제 사용 간의 불일치(train-test mismatch)가 발생함. 이를 완화하기 위해 세 가지를 섞어 사용함.

→ 모델이 [MASK]에만 의존하지 않고 문장 전체의 문맥을 고르게 학습하게 만들기 위함

2. NSP (Next Sentence Prediction, 다음 문장 예측)

두 문장이 주어졌을 때 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업.

→ 문장 간의 관계와 의미적 유사성을 학습함.

특수 토큰

BERT는 입력 시퀀스에 세 가지 특수 토큰을 사용함.

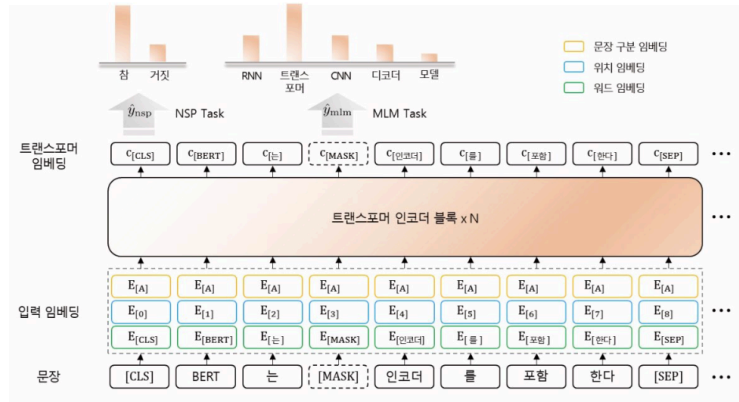


그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

입력 구조 e.g., [CLS] 문장-1 [SEP] 문장-2 [MASK] [SEP]

- **[CLS]:** 문장 맨 앞에 삽입. 문장 분류 작업(긍/부정, 개체 분류 등)에 필요한 정보를 담음
- **[SEP]:** 두 개 이상의 문장을 구분하기 위해 사용. 모델이 각 문장을 독립적으로 인식하게 함
- **[MASK]:** 임의로 선택된 단어를 가리는 토큰. MLM 학습에서 실제 단어 예측에 사용됨

입력 임베딩 구조

각 토큰의 임베딩은 세 가지를 합산해 구성됨:

- **워드 임베딩:** 단어 자체의 의미
- **위치 임베딩:** 시퀀스 내 토큰 위치
- **문장 구분 임베딩:** 어느 문장에 속하는지 구분

Fine-tuning

사전 학습 후 미세 조정을 통해 다양한 downstream 작업에 적용 가능: 문장 분류, 감성 분석, 질문 응답, 기계 번역 등. 해당 작업에 맞는 계층을 추가하고 손실 함수를 정의해 학습함.

- 분류 문제: [CLS] 토큰 벡터 사용
- 토큰별 예측이 필요한 경우: 모든 토큰 벡터 사용

모델 실습

```
# 7.15 네이버 영화 리뷰 데이터 불러오기
import numpy as np
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
df = pd.DataFrame(corpus.test).sample(20000, random_state=42)
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(train.head(5).to_markdown())
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

▼ 실행 결과

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을
손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다.
해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 description 을 참고,
해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```
# Description
Author : e9t@github
Repository : https://github.com/e9t/nsmc
References : www.lucypark.kr/docs/2015-pyconkr/#39
```

Naver sentiment movie corpus v1.0
This is a movie review dataset in the Korean language.
Reviews were scraped from Naver Movies.

The dataset construction is based on the method noted in
[Large movie review dataset][^1] from Maas et al., 2011.

[^1]: <http://ai.stanford.edu/~amaas/data/sentiment/>

```
# License
CC0 1.0 Universal (CC0 1.0) Public Domain Dedication
Details in https://creativecommons.org/publicdomain/zero/1.0/
```

```
...
| 18010 | 징키스칸이란 소재를 가지고 이것밖에 못만드나 | 0 |
Training Data Size : 12000
Validation Data Size : 4000
Testing Data Size : 4000
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#).
[/opt/homebrew/Caskroom/miniforge/base/envs/dl-euron/lib/python3.10/site-packag](#)
return bound(*args, **kwargs)

7.16 BERT 입력 텐서 생성

```
import torch
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

def make_dataset(data, tokenizer): # device 인자 제거
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest", # 배치 내 가장 긴 시퀀스 기준으로 패딩
        truncation=True, # 모델 최대 길이(512) 초과 시 잘라냄
        return_tensors="pt" # PyTorch 텐서로 반환
    )
    input_ids = tokenized["input_ids"] # device로 안 올림
    # input_ids: 각 토큰을 정수 ID로 변환한 텐서 ([CLS], 단어들, [SEP] 포함)

    attention_mask = tokenized["attention_mask"]
    # attention_mask: 실제 토큰(1)과 패딩 토큰(0)을 구분 - 패딩에 attention 안 주도록

    labels = torch.tensor(data.label.values, dtype=torch.long)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 5
batch_size = 32
```

```
device = "mps" if torch.backends.mps.is_available() else "cpu" # cuda → mps로 변경 (Apple Silicon)

tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased", # 다국어 BERT, 한국어 포함
    do_lower_case=False # 대소문자 구분 유지 (cased 모델이므로 False)
)

train_dataset = make_dataset(train, tokenizer) # device 인자 제거
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)
# RandomSampler: 훈련 시 매 에폭마다 데이터를 섞어 과적합 방지

valid_dataset = make_dataset(valid, tokenizer)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)
# SequentialSampler: 검증/테스트는 순서대로 - 결과 재현성 확보

test_dataset = make_dataset(test, tokenizer)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])
```

```
# AdamW: BERT 파인튜닝 표준 옵티마이저 (Adam + weight decay)
# lr=1e-5: BERT 파인튜닝 권장 학습률 (너무 크면 사전학습 가중치 손상)
# eps=1e-8: 수치 안정성을 위한 값 (0으로 나누는 것 방지)
```

▼ 실행 결과

```
model.safetensors: 100%  714M/714M [04:56<00:00, 3.83MB/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-multilingual-cased and are newly created during the training process. They were either initialized from random values or from a module specific to this checkpoint (e.g. output layer). This is expected. You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

```
# BERT 모델 구조 확인 - model 변수를 덮어쓰지 않도록 별도 변수 사용
from transformers import BertModel

bert_inspect = BertModel.from_pretrained(pretrained_model_name_or_path="bert-base-multilingual-cased")

for main_name, main_module in bert_inspect.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("  L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("    L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("      L", sssub_name)
```

▼ 실행 결과

```
transformer
  L wte
  L wpe
  L drop
  L h
    L 0
      L ln_1
      L attn
      L ln_2
      L mlp
    L 1
      L ln_1
      L attn
      L ln_2
      L mlp
    L 2
      L ln_1
      L attn
      L ln_2
      L mlp
    L 3
      L ln_1
      L attn
      L ln_2
      L mlp
    ...
      L ln_2
      L mlp
  L ln_f
  lm_head
```

```

# 모델 학습 및 검증
import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    # argmax: 각 샘플에서 가장 높은 확률의 클래스 인덱스 선택
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

def train(model, optimizer, dataloader):
    model.train() # 드롭아웃, 배치정규화 등 학습 모드 활성화
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        input_ids = input_ids.to(device) # 루프 안에서 올리기
        attention_mask = attention_mask.to(device)
        labels = labels.to(device)

        # MPS 안정성을 위해 명시적으로 contiguous 메모리 보장
        input_ids = input_ids.contiguous()
        attention_mask = attention_mask.contiguous()
        labels = labels.contiguous()

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels # labels 전달 시 내부에서 자동으로 loss 계산
        )
        loss = outputs.loss
        train_loss += loss.item()

        optimizer.zero_grad() # 이전 배치의 gradient 초기화
        loss.backward() # 역전파
        optimizer.step() # 가중치 업데이트

    train_loss = train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad(): # 평가 시 gradient 계산 비활성화 (메모리 절약)
        model.eval() # 드롭아웃 비활성화, 평가 모드
        criterion = nn.CrossEntropyLoss()
        val_loss, val_accuracy = 0.0, 0.0

        for input_ids, attention_mask, labels in dataloader:
            input_ids = input_ids.to(device) # 루프 안에서 올리기
            attention_mask = attention_mask.to(device)
            labels = labels.to(device)

            input_ids = input_ids.contiguous()
            attention_mask = attention_mask.contiguous()
            labels = labels.contiguous()

```

```

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        logits = outputs.logits
        # logits: 소프트맥스 이전의 raw 예측값 (각 클래스별 점수)

        loss = criterion(logits, labels)
        logits = logits.detach().cpu().numpy()
        label_ids = labels.to("cpu").numpy()
        accuracy = calc_accuracy(logits, label_ids)

        val_loss += loss
        val_accuracy += accuracy

    val_loss = val_loss / len(dataloader)
    val_accuracy = val_accuracy / len(dataloader)
    return val_loss, val_accuracy

best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Accuracy {val_accuracy:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "../models/BertForSequenceClassification.pt")
        # state_dict: 모델 가중치만 저장 (구조 제외) - 파인튜닝 결과 보존
        print("Saved the model weights")

```

▼ 실행 결과

```

Epoch 1: Train Loss: 0.5607 Val Loss: 0.4748 Val Accuracy 0.7678
Saved the model weights
Epoch 2: Train Loss: 0.4061 Val Loss: 0.4188 Val Accuracy 0.8133
Saved the model weights
Epoch 3: Train Loss: 0.3237 Val Loss: 0.4165 Val Accuracy 0.8180
Saved the model weights
Epoch 4: Train Loss: 0.2522 Val Loss: 0.4429 Val Accuracy 0.8150
Epoch 5: Train Loss: 0.1860 Val Loss: 0.5550 Val Accuracy 0.8183

```

```

# BERT 모델 평가
model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    num_labels=2,
    torch_dtype=torch.float32 # MPS float32 명시
).to(device) # device = "mps" (Apple Silicon)

model.load_state_dict(torch.load(

```



```

    "../models/BertForSequenceClassification.pt",
    map_location=device # 저장된 가중치를 mps 장치로 로드
))
# load_state_dict: 저장된 가중치를 현재 모델 구조에 주입

test_loss, test_accuracy = evaluation(model, test_dataloader)
print(f"Test Loss : {test_loss:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")

```

▼ 실행 결과

Test Loss : 0.4088
Test Accuracy : 0.8207

BART (Bidirectional Auto-Regressive Transformer)

- 2019년 메타 FAIR 연구소가 발표한 트랜스포머 기반 모델.

구조

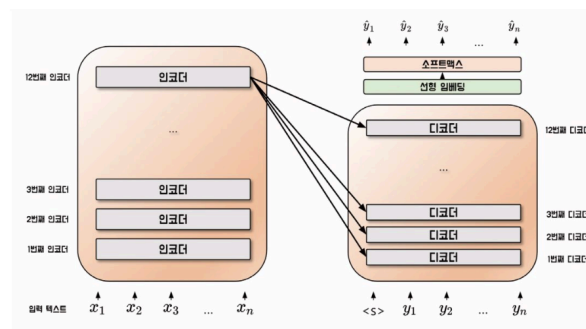


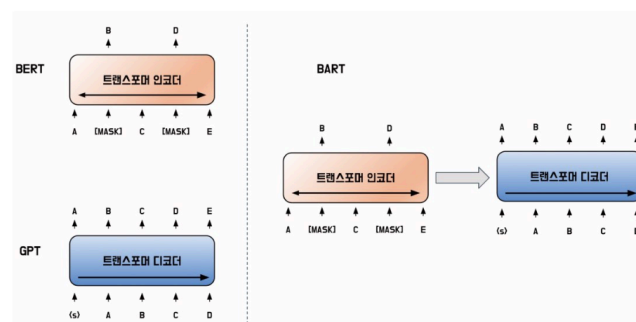
그림 7.16 BART 모델 구조

BERT의 인코더 + GPT의 디코더를 결합한 **Seq2Seq(Sequence-to-Sequence)** 구조.

- **인코더**: 양방향으로 입력 문장을 처리 → 문장 전체 맥락 이해 및 단어 간 상호작용 파악
- **디코더**: 이전 토큰들을 입력으로 받아 다음에 올 토큰을 순차적으로 생성

트랜스포머와 유사하지만 차이점이 있음. 트랜스포머는 인코더의 모든 계층과 디코더의 모든 계층 사이에서 어텐션 연산을 수행하는 반면, BART는 **인코더의 마지막 계층과 디코더의 각 계층 사이에서만** 어텐션 연산을 수행함 → 정보 전달 최적화 + 메모리 사용량 절감.

사전 학습 방법



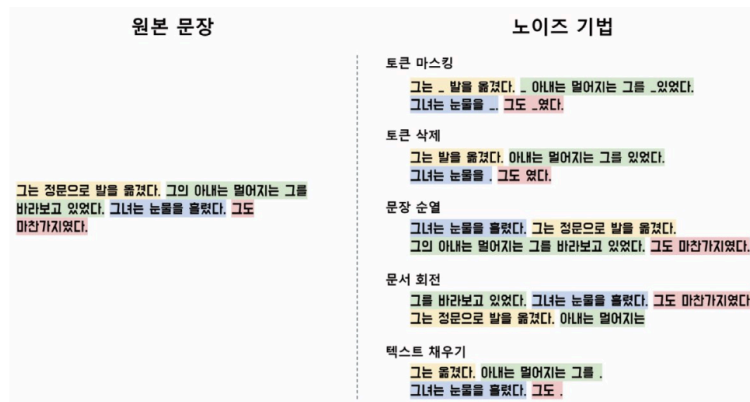


그림 7.17 BART 노이즈 기법

1. 토큰 마스크 (Token Masking)

BERT의 MLM과 동일. 일부 토큰을 [MASK]로 치환. 어떤 단어가 문장 내에서 중요한 역할을 하는지 학습하고 문맥 이해 및 중요 정보 추출 능력이 향상됨.

2. 토큰 삭제 (Token Deletion)

토큰을 치환하는 것이 아니라 아예 삭제. 마스크와 달리 어떤 위치의 토큰이 삭제되었는지도 맞춰야 함. 불필요한 정보를 필터링하는 능력이 생기고, 모델의 일반화 성능이 향상됨.

3. 문장 순열 (Sentence Permutation)

마침표(.)를 기준으로 문장을 나눈 뒤 순서를 섞음. 모델은 원래의 문장 순서를 복원해야 함. 단어 간 연결 관계를 더 잘 파악하게 되고, 다른 순서로 입력이 주어져도 잘 처리할 수 있게 됨.

4. 문서 회전 (Document Rotation)

임의의 토큰으로 문서가 시작하도록 함. 문장의 순서는 유지하되 시작 위치만 바꿈. 모델은 원래의 시작 토큰을 맞춰야 하며, 이를 통해 문서의 시작점을 인식하는 능력이 생김.

5. 텍스트 채우기 (Text Infilling)

여러 토큰을 하나의 구간(span)으로 묶고, 그 구간을 단 하나의 [MASK] 토큰으로 대체. 구간 길이는 **포아송 분포($\lambda=3$)**로 결정되며, 길이가 0이면 해당 위치에 변환된 토큰이 없음을 의미함.

토큰 마스크와의 차이점: 토큰 마스크는 1개 토큰 → 1개 [MASK]인 반면, 텍스트 채우기는 **N개 토큰 → 1개 [MASK]**로 대체. 모델은 연속된 마스크 토큰을 복구하되, 실제로 마스크되지 않은 토큰도 구분해야 함. 이를 통해 누락된 단어를 예측하는 능력이 향상되고 입력 문장을 더 잘 이해할 수 있게 됨.

미세 조정 방법

인코더와 디코더에 **다른 문장 구조로 입력**해야 함 (작업별로 구성 방식이 다름).

1. 문장 분류

입력 문장을 인코더와 디코더에 **동일하게** 입력. 디코더의 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 사용. BERT의 [CLS] 토큰과 비슷하지만, BART는 전체 입력과 어텐션 연산이 적용된 은닉 상태를 사용한다는 점이 다름.

2. 토큰 분류

문장 분류와 동일하게 인코더·디코더에 같은 문장을 입력. 디코더의 **각 시점별** 마지막 은닉 상태를 토큰 분류기의 입력값으로 사용. 전체 입력과 디코더의 각 시점별 은닉 상태와의 어텐션 연산을 수행.

3. 문장 생성

BART는 디코더가 있으므로 BERT가 하지 못했던 문장 생성 작업 수행 가능. 추상적 질의응답(Abstractive QA), 문장 요약(Summarization) 등에 특히 적합. 사전 학습 방식(노이즈 복원)과 유사하기 때문에 뛰어난 성능을 보임.

4. 기계 번역

사전 학습된 BART 인코더에 **번역용 인코더를 추가**해 작업 수행.

- 추가된 인코더: 기존 단어 사전 사용 안 함 (새 언어의 단어 처리)
- 디코더: 사전 학습된 가중치와 단어 사전 그대로 사용

학습은 두 단계로 진행됨:

- 1단계: 새로 추가된 인코더의 가중치 + 위치 임베딩 + 첫 번째 인코더 층의 셀프 어텐션 입력 행렬 가중치만 학습
- 2단계: 모든 신경망 가중치를 작은 반복으로 학습

모델 실습

```
# 7.18 뉴스 요약 데이터셋 불러오기
import numpy as np
from datasets import load_dataset

news = load_dataset("argilla/news-summary", split="test")
df = news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])

train_data, valid_data, test_data = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)
# train/valid/test 변수명 충돌 방지를 위해 _data 접미사 사용

print(f"Source News : {train_data.text.iloc[0][:200]}")
print(f"Summarization : {train_data.prediction.iloc[0][:50]}")
print(f"Training Data Size : {len(train_data)}")
print(f"Validation Data Size : {len(valid_data)}")
print(f"Testing Data Size : {len(test_data)}")
```

▼ 실행 결과

```
README.md: 2.02k/? [00:00<00:00, 190kB/s]

data/train-00000-of-00001- 1.54M/1.54M [00:02<00:00, 7.65MB/s]
ebc48879f34571(...): 100%

data/test-00000-of-00001- 31.7M/31.7M [00:16<00:00, 158MB/s]
6227bd8eb10a9b5(...): 100%

Generating train split: 100% 1000/1000 [00:00<00:00, 29816.83 examples/s]

Generating test split: 100% 20417/20417 [00:00<00:00, 261163.85 examples/s]

Source News : DANANG, Vietnam (Reuters) - Russian President Vladimir Putin said on Saturday he had a normal
Summarization : Putin says had useful interaction with Trump at Vi
Training Data Size : 3000
Validation Data Size : 1000
Testing Data Size : 1000
```

```
# BART 입력 텐서 생성
import os
os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1" # MPS 미지원 연산 CPU fallback
```

```

import torch
from transformers import BartTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from torch.nn.utils.rnn import pad_sequence

def make_dataset(data, tokenizer):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        max_length=512,          # 입력 최대 길이 명시
        return_tensors="pt"
    )
    labels = []
    input_ids = tokenized["input_ids"]
    attention_mask = tokenized["attention_mask"]

    for target in data.prediction:
        label = tokenizer.encode(
            target,
            return_tensors="pt",
            truncation=True,      # 추가: label도 truncation 적용
            max_length=128       # 요약문 최대 길이 제한
        ).squeeze()
        labels.append(label)

    labels = pad_sequence(labels, batch_first=True, padding_value=-100)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 3
batch_size = 8
device = "cpu"

tokenizer = BartTokenizer.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
    # 영어 뉴스 요약에 적합한 BART base 모델
)

train_dataset = make_dataset(train_data, tokenizer)      # train → train_data (변수명
충돌 방지)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid_data, tokenizer)      # valid → valid_data
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test_data, tokenizer)        # test → test_data

```



```

model
├── shared
├── encoder
│   ├── embed_tokens
│   ├── embed_positions
│   ├── layers
│   │   ├── 0
│   │   ├── 1
│   │   ├── 2
│   │   ├── 3
│   │   ├── 4
│   │   └── 5
│   └── layernorm_embedding
├── decoder
│   ├── embed_tokens
│   ├── embed_positions
│   ├── layers
│   │   ├── 0
│   │   ├── 1
│   │   ├── 2
│   │   ├── 3
│   │   ├── 4
│   │   └── 5
│   └── layernorm_embedding
└── lm_head

```

```

# BART 모델 학습 및 평가
import numpy as np
import evaluate
from torch import nn

rouge_score = evaluate.load("rouge", tokenizer=tokenizer)

def calc_rouge(preds, labels):
    preds = preds.argmax(axis=-1)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
    rouge2 = rouge_score.compute(
        predictions=decoded_preds,
        references=decoded_labels
    )
    return rouge2["rouge2"]

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        # cpu이므로 .to(device) 불필요하지만 device 변경 시 호환성을 위해 유지
        input_ids = input_ids.to(device)
        attention_mask = attention_mask.to(device)
        labels = labels.to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,

```

```

        labels=labels
    )
    loss = outputs.loss
    train_loss += loss.item()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

return train_loss / len(dataloader)

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss, val_rouge = 0.0, 0.0

        for input_ids, attention_mask, labels in dataloader:
            input_ids = input_ids.to(device)
            attention_mask = attention_mask.to(device)
            labels = labels.to(device)

            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )
            logits = outputs.logits
            loss = outputs.loss

            logits = logits.detach().numpy() # cpu이므로 .cpu() 불필요
            label_ids = labels.numpy() # cpu이므로 .to("cpu") 불필요
            rouge = calc_rouge(logits, label_ids)

            val_loss += loss
            val_rouge += rouge

        return val_loss / len(dataloader), val_rouge / len(dataloader)

best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_rouge = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val ROUGE2: {val_rouge:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "../models/BartForConditionalGeneration.pt")
        print("Saved the model weights")

```

▼ 실행 결과

```
Epoch 1: Train Loss: 2.1573 Val Loss: 1.8756 Val ROUGE2: 0.2621
Saved the model weights
Epoch 2: Train Loss: 1.5957 Val Loss: 1.8786 Val ROUGE2: 0.2571
Epoch 3: Train Loss: 1.2374 Val Loss: 1.9737 Val ROUGE2: 0.2571
```

```
# 7.22 BART 모델 평가
model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
).to(device)

model.load_state_dict(torch.load(
    "../models/BartForConditionalGeneration.pt",
    map_location=device # cpu로 로드
))

test_loss, test_rouge_score = evaluation(model, test_dataloader)
print(f"Test Loss : {test_loss:.4f}")
print(f"Test ROUGE-2 Score : {test_rouge_score:.4f}")
```

▼ 실행 결과

```
# 7.23 문장 요약문 비교
from transformers import pipeline

summarizer = pipeline(
    task="summarization",
    model=model,
    tokenizer=tokenizer,
    max_length=54,
    device="cpu"
)

for index in range(5):
    news_text = test_data.text.iloc[index] # test → test_data (변수명 충돌 방지)
    summarization = test_data.prediction.iloc[index]
    predicted_summarization = summarizer(news_text)[0]["summary_text"]
    print(f"정답 요약문 : {summarization}")
    print(f"모델 요약문 : {predicted_summarization}\n")
```

▼ 실행 결과

ELECTRA

- 2020년 구글이 발표한 트랜스포머 기반 모델.

핵심 아이디어

기존 BERT 계열은 MLM 방식으로 일부 토큰을 [MASK]로 대체 후 복원하는 방식으로 학습. ELECTRA는 마스킹 대신 **생성자(Generator)**와 **판별자(Discriminator)**를 사용해 GAN과 유사한 방식으로 사전 학습.

GAN과의 차이점

GAN과 유사하지만 세 가지 차이점이 있음:

1. **정확히 예측한 토큰 처리:** 생성자가 원래 토큰을 정확히 예측한 경우, GAN은 생성된 것으로 간주하지만 ELECTRA는 원본 토큰으로 간주함
2. **학습 방식:** GAN은 생성 모델과 판별 모델이 적대적으로 학습하지만, ELECTRA의 생성 모델은 MLM으로 학습하고 판별 모델은 RTD로 학습함
3. **입력 방식:** GAN은 완전한 노이즈 벡터를 입력받아 생성하지만, ELECTRA의 생성 모델은 일부 토큰이 마스크 처리된 텍스트를 입력으로 받음

BERT 대비 장점

- GAN 방식으로 더 효율적인 학습 → 대규모 데이터셋에서 더 빠른 학습
- 매개변수 수가 더 적어 모델 크기가 작고, 더 빠른 실행 + 더 적은 메모리 사용
- 생성 모델을 통해 다양한 자연어 생성 작업에서 더 자연스러운 문장 생성

사전 학습 방법 : RTD (Replaced Token Detection)

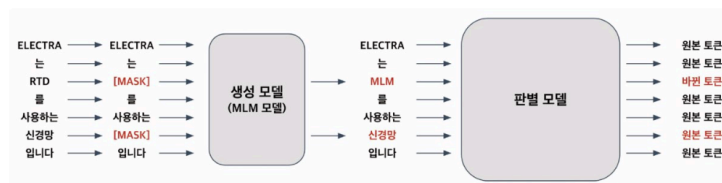


그림 7.20 ELECTRA 모델 학습 방법

생성자 (Generator) — BERT의 MLM 모델과 동일

- 입력 문장의 15%를 마스크 처리
- 마스크 처리된 토큰이 원래 어떤 토큰이었는지 예측하며 학습

판별자 (Discriminator)

- 생성자가 복원한 결과값을 입력받아 각 토큰이 원본 토큰인지 바뀐 토큰인지 판별
- 이 학습 방법을 **RTD(Replaced Token Detection)** 라고 함

가중치 공유 전략

생성 모델과 판별 모델은 모두 트랜스포머 인코더 구조. 같은 계층 수로 구성되면 가중치를 공유할 수 있어 빠른 학습이 가능하지만, 완전히 공유하면 생성 모델 성능이 너무 높아져 판별 모델이 학습할 수 없게 됨.

따라서 ELECTRA는:

- 생성 모델 크기를 판별 모델의 **1/2 ~ 1/4** 로 설정
 - 모든 가중치 공유 대신 **임베딩 계층의 가중치만 공유**
- 판별 모델이 학습하기 적합한 생성 모델을 유지하면서 효율적으로 학습 가능.

파인튜닝

사전 학습 완료 후 생성 모델은 사용하지 않고 **판별 모델만** 사용해 다운스트림 작업 수행. 판별 모델은 트랜스포머 인코더로 구성되어 있으며 BERT와 동일한 구조. 파인튜닝 방식도 BERT와 동일.

모델 실습

```
# 7.24 네이버 영화 리뷰 데이터셋 전처리
import torch
from transformers import ElectraTokenizer
```

```

from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

def make_dataset(data, tokenizer):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"]
    attention_mask = tokenized["attention_mask"]

    labels = torch.tensor(data.label.values, dtype=torch.long) # data.label (s 없음)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 5
batch_size = 32
device = "cpu" # MPS 대신 cpu 사용

tokenizer = ElectraTokenizer.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    # 한국어 ELECTRA 판별자 모델 - 파인튜닝에는 판별자 사용
    do_lower_case=False, # 대소문자 구분 유지
)

train_dataset = make_dataset(train_data, tokenizer) # train → train_data (변수명 충돌 방지)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)
# RandomSampler: 훈련 시 매 에폭마다 데이터를 섞어 과적합 방지

valid_dataset = make_dataset(valid_data, tokenizer) # valid → valid_data
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)
# SequentialSampler: 검증/테스트는 순서대로 - 결과 재현성 확보

test_dataset = make_dataset(test_data, tokenizer) # test → test_data
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])

```

▼ 실행 결과

```
(tensor([ 2, 6511, 14347, 4087, 4665, 4112, 2924, 4806, 16, 3809,
4309, 4275, 16, 3201, 4376, 2891, 4139, 4212, 4007, 6557,
4200, 5, 5, 3, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0]), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0]), tensor(1))
```

```
# 7.25 KoELECTRA 모델 선언
from torch import optim
from transformers import ElectraForSequenceClassification

model = ElectraForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    # 사전 학습된 KoELECTRA 판별자 위에 분류용 linear layer를 추가한 파인튜닝 전용 클래스
    num_labels=2 # 감성 분류: 긍정(1) / 부정(0)
).to(device) # device = "cpu"

optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
# AdamW: ELECTRA 파인튜닝 표준 옵티마이저 (Adam + weight decay)
# lr=1e-5: BERT/ELECTRA 파인튜닝 권장 학습률
# eps=1e-8: 수치 안정성을 위한 값 (0으로 나누는 것 방지)
```

```
# KoELECTRA 모델 구조 확인
koelectra_inspect = ElectraForSequenceClassification.from_pretrained(pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator")

for main_name, main_module in koelectra_inspect.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("└", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("└└", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("└└└", sssub_name)
```

▼ 실행결과

```

Some weights of ElectraForSequenceClassificati
You should probably TRAIN this model on a down
electra
└─ embeddings
│   └─ word_embeddings
│   └─ position_embeddings
│   └─ token_type_embeddings
│   └─ LayerNorm
│   └─ dropout
└─ encoder
    └─ layer
        └─ 0
        └─ 1
        └─ 2
        └─ 3
        └─ 4
        └─ 5
        └─ 6
        └─ 7
        └─ 8
        └─ 9
        └─ 10
        └─ 11
└─ classifier
    └─ dense
    └─ activation
    └─ dropout
    └─ out_proj

```

```

# 모델 학습 및 검증
import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    # argmax: 각 샘플에서 가장 높은 확률의 클래스 인덱스 선택
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

def train(model, optimizer, dataloader):
    model.train() # 드롭아웃, 배치정규화 등 학습 모드 활성화
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        input_ids = input_ids.to(device)
        attention_mask = attention_mask.to(device)
        labels = labels.to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels # labels 전달 시 내부에서 자동으로 loss 계산
        )
        loss = outputs.loss
        train_loss += loss.item()

        optimizer.zero_grad() # 이전 배치의 gradient 초기화
        loss.backward() # 역전파
        optimizer.step() # 가중치 업데이트

    train_loss = train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad(): # 평가 시 gradient 계산 비활성화 (메모리 절약)
        model.eval() # 드롭아웃 비활성화, 평가 모드

```

```

criterion = nn.CrossEntropyLoss()
val_loss, val_accuracy = 0.0, 0.0

for input_ids, attention_mask, labels in dataloader:
    input_ids = input_ids.to(device)
    attention_mask = attention_mask.to(device)
    labels = labels.to(device)

    outputs = model(
        input_ids=input_ids,
        attention_mask=attention_mask,
        labels=labels
    )
    logits = outputs.logits
    # logits: 소프트맥스 이전의 raw 예측값 (각 클래스별 점수)

    loss = criterion(logits, labels)
    logits = logits.detach().numpy() # cpu이므로 .cpu() 불필요
    label_ids = labels.numpy() # cpu이므로 .to("cpu") 불필요
    accuracy = calc_accuracy(logits, label_ids)

    val_loss += loss
    val_accuracy += accuracy

val_loss = val_loss / len(dataloader)
val_accuracy = val_accuracy / len(dataloader)
return val_loss, val_accuracy

best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Accuracy {val_accuracy:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "../models/ElectraForSequenceClassification.pt")
        # state_dict: 모델 가중치만 저장 (구조 제외) - 파인튜닝 결과 보존
        print("Saved the model weights")

```

```

# 모델 평가 결과
from transformers import ElectraForSequenceClassification

model = ElectraForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    num_labels=2
).to(device) # device = "cpu"

model.load_state_dict(torch.load(
    "../models/ElectraForSequenceClassification.pt",
    map_location=device # 저장된 가중치를 cpu로 로드

```

```

))
# load_state_dict: 저장된 가중치를 현재 모델 구조에 주입

test_loss, test_accuracy = evaluation(model, test_dataloader)
print(f"Test Loss : {test_loss:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")

```

T5 (Text-to-Text Transfer Transformer)

- 2019년 구글이 발표한 트랜스포머 기반 자연어 처리 딥러닝 모델. GLUE, SuperGLUE, CNN/DM 등의 데이터셋에서 SOTA를 달성함.

핵심 특징: Text-to-Text 구조

기존 모델은 입력을 벡터/행렬로 변환 후 클래스나 일부 값을 출력하는 방식. T5는 **입력과 출력을 모두 토큰 시퀀스(텍스트)로 처리**함. 따라서 입출력 형태를 자유롭게 다룰 수 있고, 구조상 유연성과 확장성이 뛰어나 새로운 작업에도 쉽게 적용 가능.

대표 학습 작업: 문장 번역, 요약, 질의응답, 텍스트 분류 등.

구조

인코더-디코더 구조. 학습 데이터셋은 원본 문장과 대상 문장을 사용함. **C4(Colossal Clean Crawled Corpus)** 데이터셋으로 사전 학습됨.

사전 학습 방법

비지도 학습 방식으로 입력 문장의 일부 구간을 마스킹해 처리:

- 입력 시퀀스: 마스킹된 구간을 **센티널 토큰(Sentinel Token)**으로 대체
- 출력 시퀀스: 실제 마스킹된 토큰과 센티널 토큰의 연결로 구성

센티널 토큰: 문장마다 유일한 마스크 토큰. `<extra_id_0>`, `<extra_id_1>` 형태로 0~99까지 100개의 기본값을 사용.

예시: '인코더-디코더 모델 구조'에서 '인코더'와 '디코더'를 마스킹할 경우

- 입력: [`<extra_id_0>`, -, `<extra_id_1>`, 모델 구조]
- 출력: [인코더, `<extra_id_0>`, 디코더, `<extra_id_1>`]

미세 조정 (Fine-tuning)

지도 학습 방식으로 학습. 번역, 언어 수용성, 문장 유사도, 문서 요약 등 다양한 작업에 활용.

문장이 인코딩되기 전에 **작업 토큰을 문장 앞에 추가**함:

- 번역: `'translate English to German:'`
- 요약: `'summarize:'`

작업 토큰도 함께 학습하므로 다양한 자연어 처리 작업에서 높은 성능을 발휘함.

모델 실습

```

import numpy as np
from datasets import load_dataset

news=load_dataset("argilla/news-summary", split="test")
df=news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["text"]="summarize: " + df["text"] # summarize : 요약 작업임을 모델에 알리기
df["prediction"]=df["prediction"].map(lambda x: x[0]["text"])
train, valid, test=np.split(

```

```

        df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
    )

    print(f"Source News: {train.text.iloc[0][:200]}")
    print(f"Summarization: {train.prediction.iloc[0][:50]}")

```

```

import torch
from transformers import T5Tokenizer  ### 변경됨 ###
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from torch.nn.utils.rnn import pad_sequence

### 변경됨 ###
def make_dataset(data, tokenizer, device):
    source = tokenizer(
        text=data.text.tolist(),
        padding="max_length",
        max_length=128,
        pad_to_max_length=True,
        truncation=True,
        return_tensors="pt"
    )

    target = tokenizer(
        text=data.prediction.tolist(),
        padding="max_length",
        max_length=128,
        pad_to_max_length=True,
        truncation=True,
        return_tensors="pt"
    )

    source_ids = source["input_ids"].squeeze().to(device)
    source_mask = source["attention_mask"].squeeze().to(device)
    target_ids = target["input_ids"].squeeze().to(device)
    target_mask = target["attention_mask"].squeeze().to(device)
    return TensorDataset(source_ids, source_mask, target_ids, target_mask)

epochs = 3
batch_size = 8
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = T5Tokenizer.from_pretrained(
    pretrained_model_name_or_path="t5-small"
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloadner(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloadner(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloadner(test_dataset, SequentialSampler, batch_size)

```

```
print(next(iter(train_dataloader)))
print(tokenizer.convert_ids_to_tokens(21603))
print(tokenizer.convert_ids_to_tokens(10))
```

```
from torch import optim
from transformers import T5ForConditionalGeneration #T5 조건부 생성 클래스

model=T5ForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="t5-small", #가장 작은 모델, 트랜스포머와 동일한 구조
).to(device)
optimizer=optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

```
# 모델 구조 출력
for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("| | L", sssub_name)
```

```
import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat=np.argmax(preds, axis=1).flatten()
    labels_flat=labels.flatten()
    return np.sum(pred_flat==labels_flat) / len(labels_flat)

def train(model, optimizer, dataloader):
    model.train()
    train_loss=0.0

    for source_ids, source_mask, target_ids, target_mask in dataloader:
        decoder_input_ids=target_ids[:, :-1].contiguous() #디코더에 입력될 시퀀스를 인코딩
        한 텐서(마지막 토큰 제외)
        # contiguous() : 텐서를 메모리상 연속된
        # 블록으로 저장(인접한 위치의 데이터들이 빠르게 연산 가능)
        labels=target_ids[:, 1:].clone().detach()
        labels[target_ids[:, 1:]==tokenizer.pad_token_id]=-100 #

        outputs=model(
            input_ids=source_ids,
            attention_mask=source_mask,
            decoder_input_ids=decoder_input_ids,
            labels=labels,
        )

        loss=outputs.loss
        train_loss+=loss.item()
```



```

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    train_loss=train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss=0.0

        for source_ids, source_mask, target_ids, target_mask in dataloader:
            decoder_input_ids=target_ids[:, :-1].contiguous()
            labels=target_ids[:, 1:].clone().detach()
            labels[target_ids[:, 1:]==tokenizer.pad_token_id]=-100

            outputs=model(
                input_ids=source_ids,
                attention_mask=source_mask,
                decoder_input_ids=decoder_input_ids,
                labels=labels,
            )

            loss=outputs.loss
            val_loss+=loss

        val_loss=val_loss / len(dataloader)
        return val_loss

best_loss=10000
for epoch in range(epochs):
    train_loss=train(model, optimizer, train_dataloader)
    val_loss=evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f}")

    if val_loss < best_loss:
        best_loss=val_loss
        torch.save(model.state_dict(), "../T5ForConditionalGeneration.pt")
        print("Saved the model weights")

```

```

model.eval()
with torch.no_grad():
    for source_ids, source_mask, target_ids, target_mask in test_dataloader:
        generated_ids=model.generate(
            input_ids=source_ids,
            attention_mask=source_mask,
            max_length=128,
            num_beams=3,
            repetition_penalty=2.5,
            length_penalty=1.0,
            early_stopping=True,
        )

```

```
for generated, target in zip(generated_ids, target_ids):
    pred=tokenizer.decode(
        generated, skip_special_tokens=True, clean_up_tokenization_spaces=True
    )
    actual=tokenizer.decode(
        target, skip_special_tokens=True, clean_up_tokenization_spaces=True
    )
    print("Generated Headline Text:", pred)
    print("Actual Headline Text    :", actual)
break
```